

SOFTWARE ENGINEERING

Lab 3: Component Modelling & Architectural Pattern Selection

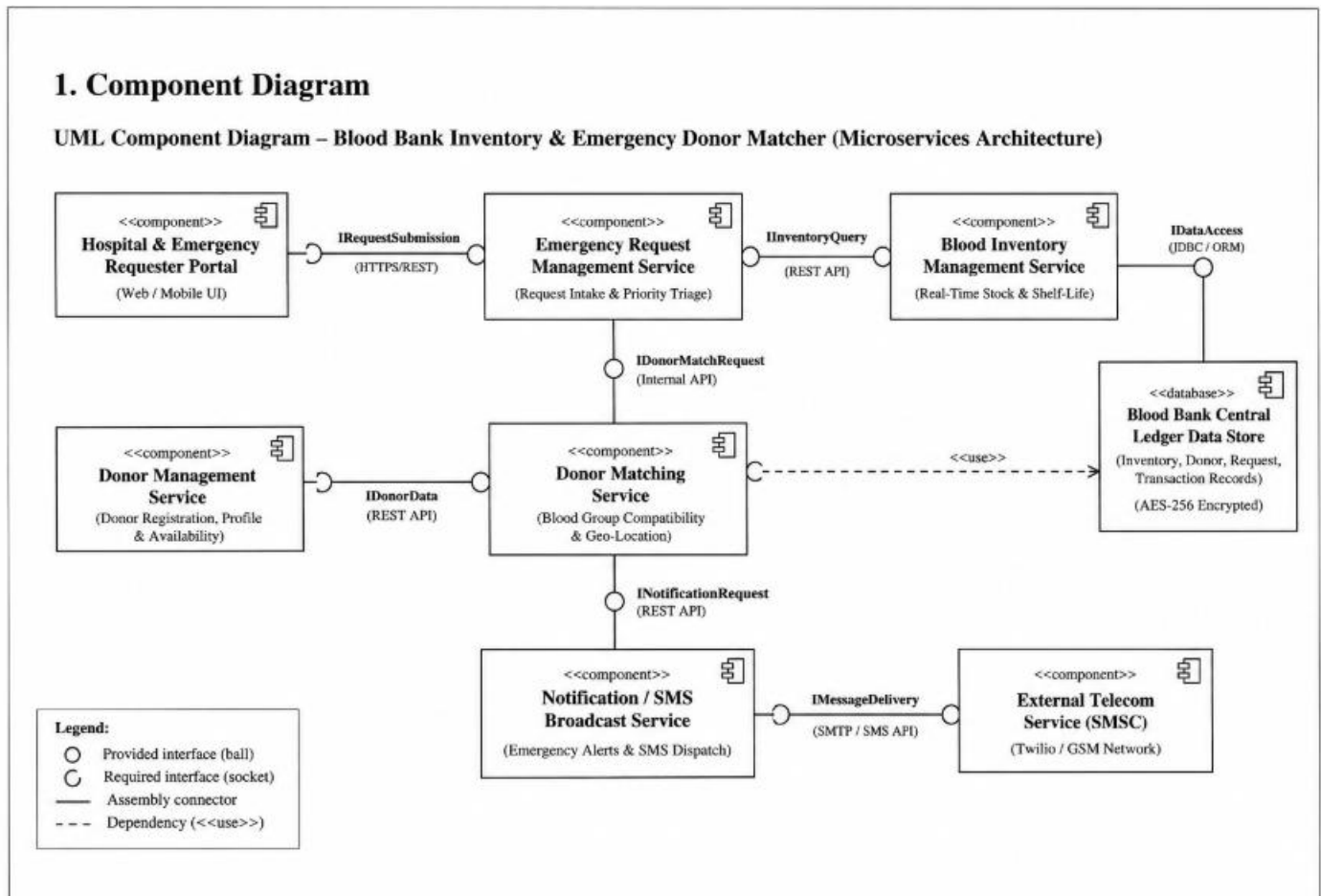
Problem Statement #12 – Healthcare & Telemedicine Blood Bank Inventory & Emergency Donor Matcher

Name: Dishanth Gowda H D

SRN: PES1UG24AM365

SEC: G

1. Component Diagram



2. Architectural Justification

Architecture Selection

We chose Microservices Architecture for the Blood Bank Inventory & Emergency Donor Matcher System.

Architectural Choice

The system is decomposed into independently deployable, loosely coupled services aligned with the system's core capabilities - emergency request triage and intake, real-time blood inventory and shelf-life tracking, geo-targeted donor compatibility matching, donor eligibility and health screening, and rapid SMS alert dispatch - each communicating through well-defined interfaces (see accompanying component diagram) rather than through a single monolithic codebase or a single centralized server.

Reason 1 - Strict Transactional Consistency and Isolated High-Concurrency Ledger Allocation (NFR-001)

In emergency trauma scenarios, multiple hospitals and healthcare facilities concurrently request scarce blood components (e.g., O-negative red blood cells or platelets). NFR-001 mandates strict transactional consistency to prevent double-allocation or race conditions where the same physical blood bag is promised to two competing emergency patients. By decoupling the Blood Inventory Management Service and backing it with an isolated transactional ledger, row-level distributed locking and atomic unit allocation can be enforced independently from high-throughput read traffic or background reporting. In a monolithic or layered architecture, intensive search queries from donor matching or portal reporting would contend for database connections and table locks against critical inventory updates, jeopardizing life-critical transactional integrity.

Reason 2 - Independent Scaling and Rapid Elasticity During Mass-Casualty Surges (FR-001)

Mass-casualty incidents, regional disasters, or sudden acute shortages create short, high-traffic windows that concentrate heavily on specific functional paths: rapid emergency request intake, donor proximity queries within a 10 km radius, and bulk dispatch of SMS notifications to hundreds of matching donors within the mandated 30-second window (FR-001). Under a microservices architecture, horizontal auto-scaling can be applied exclusively to the Emergency Request Management Service, the Donor Matching Service, and the Notification/SMS Broadcast Service without scaling the entire application stack. In contrast, a layered or client-server design would force the entire application monolith to be scaled together, wasting compute resources and introducing boot-time latency during time-critical emergency surges.

Security Advantage

Medical donor histories, blood screening records, and patient emergency requests represent sensitive Protected Health Information (PHI) subject to stringent healthcare data privacy regulations. In this microservices architecture, persistent data is strictly isolated behind the Blood Bank Central Ledger Data Store component, accessed only through an authenticated, role-scoped internal interface (IDataAccess) over TLS 1.3, with all stored blood units, donor screening logs, and audit records encrypted at rest using AES-256. Public-facing components (such as the Hospital & Emergency Requester Portal) never hold direct database credentials or query access. A compromise of an edge-facing portal service is strictly contained within its network boundary, preventing unauthorized lateral traversal to donor medical records or tampering with the blood inventory ledger.

Performance Benefit

Decoupling synchronous emergency intake from asynchronous donor matching and SMS dispatch ensures that critical hospital requests are acknowledged in milliseconds without blocking on downstream processing or external telecom latencies. While the Blood Inventory Management Service immediately verifies local stock via high-speed internal gRPC calls, secondary workflows - such as geospatial radius filtering and SMS broadcasting across external telecom networks - execute asynchronously via dedicated event queues behind their own service boundaries. This separation keeps the emergency request submission path fast and guarantees sub-second responsiveness for critical blood unit reservation while reliably achieving the FR-001 acceptance criteria of broadcasting emergency alerts to all compatible donors within 30 seconds of an emergency flag, even under peak concurrent load.